

UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ**



SPECIALIZAREA TEHNOLOGIA INFORMAȚIEI

Proiect Metode de Dezvoltare Software Harmony – Aplicație de mesagerie instantanee

**Studenti
Cherciu Cosmin-Alexandru
Stan David
Seria 26, Grupa 263**

București, 2026

Contents

1. Introducere.....	4
1.1 Prezentare generală.....	4
1.2 Specificații	4
2. Arhitectură	5
2.1 Arhitectură generală.....	5
2.2 Modelul de date	6
Descrierea entităților.....	6
Descrierea relațiilor dintre entități.....	9
Diagrama Entitate-Relație	10
Diagrama Conceptuală	11
3. Implementare backend.....	12
3.1 Overview tehnologii și tech stack.....	12
3.2 Netty	12
3.3 Protocol de comunicare	13
3.4 Comunicarea între server și client	13
Obiecte DTO.....	13
Serializare/Deserializare.....	13
3.5 Comunicarea între server și baza de date	14
3.6 Integrarea Inteligenței Artificiale (AI).....	14
Asistentul de Scriere (AI Polish) (qwen2.5:7b)	14
Recomandări Inteligente de Grupuri (gemma3:4b).....	15
3.7 Interfața cu utilizatorul și arhitectura MVP (Model-View-Presenter).....	15
Arhitectura MVP (Model-View-Presenter)	15
Structura Interfeței Grafice	16
3.8 Prezentarea interfeței grafice	17
3.9 Diagrame UML.....	21
Diagramă de secvență.....	21
Diagrama de componente	22
Diagrame use-case	23
Diagramă de use-case general	24

4. Testare	25
4.1 Tehnologii și Biblioteci	25
4.2 Teste Unitare (Unit Tests)	25
Testele pentru Obiectele de Acces ale Bazei de Date (DAO - Data Access Objects).....	25
Testele pentru Obiectele de Transfer de Date (DTO – Data Transfer Objects)....	25
Testele de Backend (Netty Server și Client).....	26
Testele de Frontend (Arhitectura MVP – Presenters).....	26
4.3 Teste de Integrare (Integration Tests)	27
DaoIntegrationTest	27
ServerIntegrationTest.....	27
ComplexFlowIntegrationTest	27
4.4 Integrare continuă (GitHub Actions)	28
5. Raport privind folosirea instrumentelor AI în timpul dezvoltării.....	29

1. Introducere

1.1 Prezentare generală

Harmony a fost proiectată ca o aplicație de mesagerie instantanee, inspirată în principal de *Discord*. Aplicația are la bază un sistem de „prietenii”. Utilizatorii își creează un cont, după care fac „cereri de prietenie” către alți utilizatori. „Prietenii” își pot trimite mesaje între ei și pot forma mesaje în grup. Peste această bază, am adăugat o funcționalitate de „interese”. Utilizatorii își pot stabili interese și, folosind un agent AI, le putem recomanda grupuri în care se discută despre subiectele respective.

1.2 Specificații

- Utilizatori
 - utilizatorii pot intra într-un cont deja existent folosind un username (unic) și o parolă
 - utilizatorii își pot crea un cont dacă nu au unul deja
 - parolele vor fi criptate
- Prietenii
 - utilizatorii pot trimite cereri de prietenie către alți utilizatori, în baza *username*-ului
 - utilizatorii pot vizualiza ceilalți useri cu care sunt prietenii
 - utilizatorii pot vizualiza cererile de prietenie pe care le-au primit
 - cererile de prietenie pot fi acceptate sau respinse
- Conversații
 - utilizatorii pot trimite mesaje către prietenii lor
 - mesajele trebuie să fie afișate instantaneu atunci când sunt primite
 - utilizatorii pot crea grupuri (group chat-uri) cu mulți utilizatori
- Mesaje
 - mesajele pot fi editate de către userul care le-a trimis
 - mesajele pot fi șterse de către userul care le-a trimis
- Interese/Subiecte
 - utilizatorii își pot asocia mai multe *interese* contului lor
 - interesele pot fi alese dintr-o listă deja existentă sau definite de utilizator
 - grupurile pot avea asociate subiecte de discuție
 - utilizatorilor li se vor recomanda grupuri cu subiecte de discuție care se încadrează în interesele lor

2. Arhitectură

2.1 Arhitectură generală

Aplicația urmărește modelul Client-Server, separând logica dedicată unui end-user cu logica ce ține de gestiunea datelor și bazei de date. Astfel, serverul poate rula și pe altă mașină, clientul comunicând cu acesta prin internet.

- Client
 - Frontend
 - prezintă informațiile către utilizator
 - definește modul de interacțiune al utilizatorului cu backend-ul aplicației
 - Backend
 - trimite cereri către server
 - procesează informațiile primite de la server și le expune către frontend
- Server
 - procesează cererile trimise de către clienți
 - gestionează baza de date
 - răspunde cu informațiile cerute și/sau face modificările necesare în baza de date
 - gestionează și/sau rulează agenții AI
- Baza de date
 - stochează toate informațiile necesare aplicației

2.2 Modelul de date

Baza de date va trebui să stocheze toate datele ce fac posibilă funcționarea aplicației. Aceasta va stoca denumirile și parolele utilizatorilor, relațiile de prietenie dintre aceștia, interesele acestora, conversațiile din care fac parte și mesajele trimise în aceste conversații.

Descrierea entităților

HM_USER

Stochează username-ul și parola utilizatorilor. Conține și un câmp momentan nefolosit de email, pentru o posibilă implementare viitoare. Username-ul este cheia primară, deci trebuie să fie unic.

Chei:

- PK – **username** – *varchar(64)* – numele și identificatorul (unic) al utilizatorului

Atribute:

- **pass** – *varchar(128)*, NN – parola utilizatorului, stocată ca hash
- **email** – *varchar(128)* – adresa de email a utilizatorului, momentan nefolosită

USER_FRIEND – tabel asociativ

Tabelul care stochează relațiile între utilizatori, facilitând sistemul de friend request. Reține dacă acesta este un request care trebuie să fie acceptat/respins sau dacă este o relație activă de prietenie

Chei:

- PK (**user1**, **user2**) – cheia primară compusă a tabelului; nu poate apărea de două ori o relație între aceiași doi useri
- FK – **user1** – *varchar(64)* – identificatorul primului user din relație
- FK – **user2** – *varchar(64)* – identificatorul celui de-al doilea user din relație

Atribute:

- **status** – *enum(outgoing, accepted)* – reține statusul relației

CHAT

Stochează informații despre conversații. Acest tabel este folosit atât pentru conversațiile între utilizatori individuali cât și pentru conversațiile în grup.

Chei:

- PK – **chatID** – *bigserial* – identificatorul unic generat automat pentru fiecare conversație

Atribute:

- *chatName* - *varchar(128)* – numele conversației, în cazul în care este un grup
- *isGroup* – *bool*, *NN* – desemnează dacă conversația este un grup sau nu
- *updated_at* – *timestampz* – data și ora la care s-a trimis ultimul mesaj în conversație

CHAT_MEMBER – tabel asociativ

Reține relațiile dintre utilizatori și conversații. Din ce conversații face parte fiecare utilizator și ce rol are în aceasta (creator, admin, membru). Pentru conversațiile unu-la-unu (de acum încolo DM - direct message), ambii utilizatori sunt membri.

Chei:

- PK – (**chatID**, **memberID**) – cheia primară compusă a tabelului; un user nu poate face parte din aceeași conversație de mai multe ori
- FK – **chatID** – *bigint* – identificatorul unic al conversației
- FK – **memberID** – *varchar(64)* – identificatorul utilizatorului

Atribute:

- *hm_role* - *enum(creator, admin, membru)*, *NN* – rolul utilizatorului în conversația respectivă
- *last_access* – *timestampz* – data și ora la care user-ul a vizualizat ultima oară conversația

HM_MESSAGE

Stochează toate mesajele trimise prin aplicație. Reține corpul mesajului, cine a trimis mesajul, conversația în care s-a trimis, momentul în care s-a trimis și tipul mesajului. Tipul mesajului nu este încă implementat, a fost gândit pentru a facilita afișarea unor diferite tipuri de mesaje în diferite moduri, spre exemplu mesaje de sistem sau mesaje de tip „user has left/joined”.

Chei:

- PK – **messID** – *bigserial* – identificatorul unic generat automat pentru fiecare mesaj
- FK – **senderID** – *varchar(64)* – identificatorul utilizatorului care a trimis mesajul
- FK – **chatID** – *bigint* – identificatorul conversației în care a fost trimis mesajul

Atribute:

- **message_content** - *text* – conținutul mesajului
- **sent_at** – *timestampz* – data și ora la care s-a trimis mesajul
- **message_type** – *enum(regular, system, join, leave)*, *NN* – desemnează tipul mesajului; implicit *regular*

HM_TOPICS

Stochează topicurile/subiectele de discuție, pentru implementarea funcționalității de interese.

Chei:

- PK – **topicID** – *bigserial* – identificatorul unic generat automat pentru fiecare topic

Atribute:

- **name** – *varchar(100)* – numele topicului
- **is_official** - *bool* – dacă topicul este adăugat de administratori sau utilizatori

USER_TOPIC – tabel asociativ

Reține asocierile dintre utilizatori și topicuri, în interfață sub numele de „interese”

Chei:

- PK – (**userID, topicID**) – cheia primară compusă a tabelului; un user nu poate avea de două ori același interes
- FK – **userID** – *varchar(64)* – identificatorul unic al utilizatorului
- FK – **topicID** – *varchar(100)* – identificatorul topicului

CHAT_TOPIC – tabel asociativ

Reține asocierile dintre conversații și topicuri.

Chei:

- PK – (**userID, chatID**) – cheia primară compusă a tabelului; un user nu poate avea de două ori același interes
- FK – **userID** – *varchar(64)* – identificatorul unic al utilizatorului
- FK – **chatID** – *bigint* – identificatorul unic al conversației

Descrierea relațiilor dintre entități

1. User – User: M(0) – M(0)
 - Descrie relațiile de prietenie
 - Un user poate fi prieten cu mai mult useri
 - Un user poate fi prietenul mai multor useri
 - Un user nu trebuie să aibă prieteni sau să fie prietenul cuiva
2. User – Chat: M(0) – M(0)
 - Un user poate face parte din mai multe chat-uri
 - La un chat pot participa mai mulți useri
 - Un user nu trebuie să facă parte dintr-un chat
 - Un chat nu trebuie neapărat să aibă useri în el
3. Chat - Message: 1(1) – M(0)
 - Un mesaj trebuie să fie asociat exact unui singur chat
 - Un chat poate conține mai multe mesaje
 - Un chat nu trebuie neapărat să conțină un mesaj

4. User – Message: 1(1) – M(0)
 - Un mesaj trebuie sa fie trimis de exact un singur user
 - Un user poate trimite mai multe mesaje
 - Un user nu trebuie neaparat sa aiba mesaje trimise
5. User – Topics: M(0) – M(0)
 - Un user poate avea asociate mai multe topicuri ca fiind interese
 - Un user nu trebuie sa aiba un topic de care este interesat
 - Un topic poate fi asociat mai multor useri
 - Un topic nu trebuie sa aiba neaparat vreun user asociat
6. Chat – Topics: M(0) – M(0)
 - Un chat poate avea mai multe topicuri asociate ca subiecte de discuție
 - Un chat nu trebuie sa aibă un topic asociat
 - Un topic poate avea mai multe chat-uri cu care este asociat
 - Un topic nu trebuie sa aibă neapărat un chat cu care să fie asociat

Diagrama Entitate-Relație

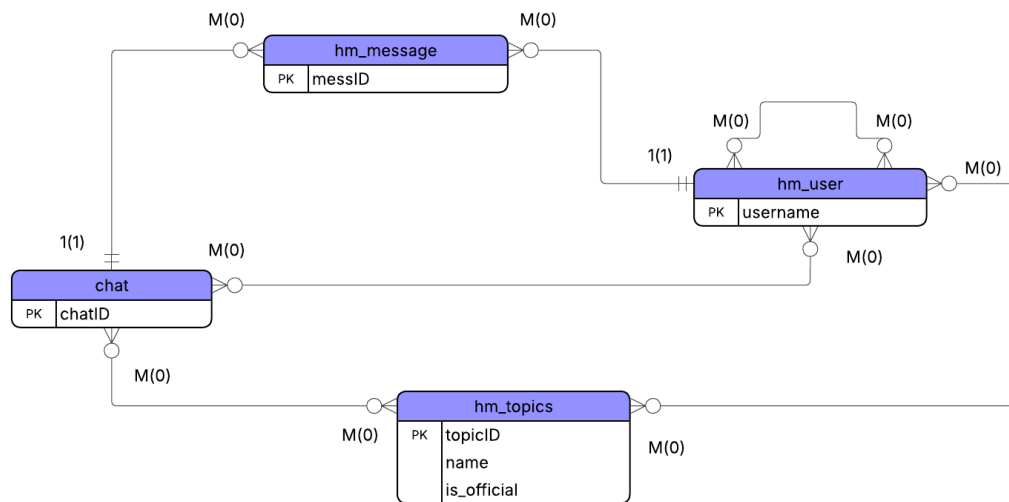
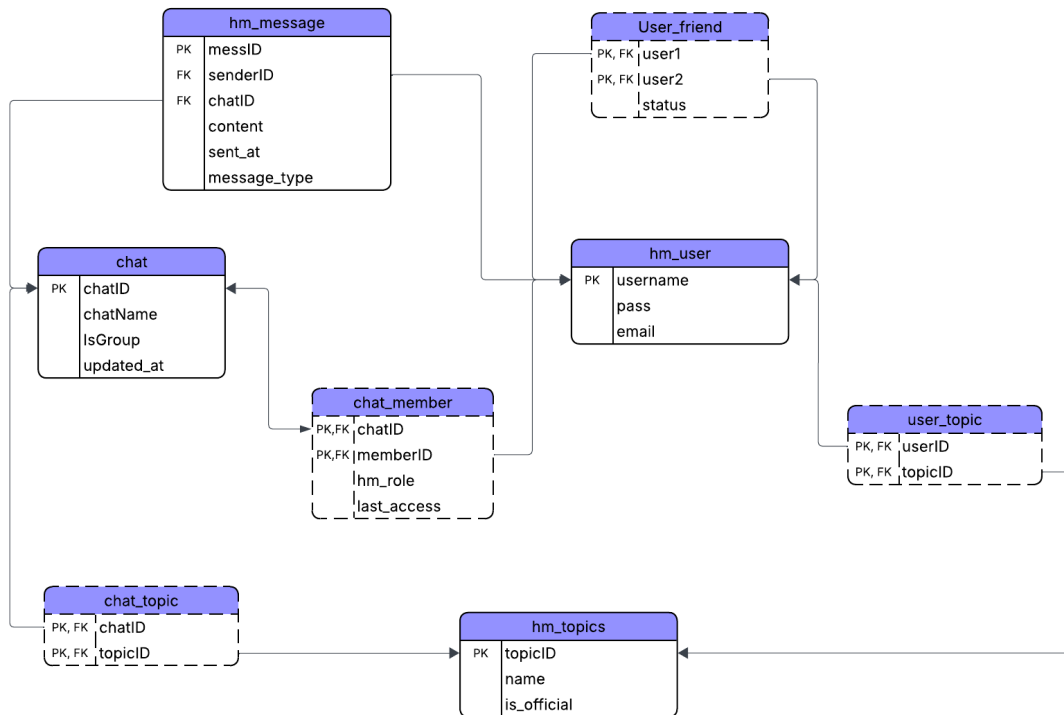


Diagrama Conceptuală



3. Implementare backend

3.1 Overview tehnologii și tech stack

- Frontend
 - Swing + FlatLaf
- Backend
 - Netty – framework pentru aplicații de networking cu NIO
 - Jackson – serializare/deserializare JSON
 - HikariCP – managementul conexiunii și operațiunilor cu baza de date
- Baza de date
 - PostgreSQL
- Serviciu Extern AI
 - Ollama

3.2 Netty

Netty reprezintă, practic vorbind, fundația proiectului. Acesta este framework-ul în jurul căruia a fost construit întregul backend al proiectului. Netty este un framework pentru limbajul de programare **Java**, destinat dezvoltării de aplicații de networking client-server cu operații I/O non-blocante (asincrone). Este gândit pentru aplicații cu numere mari de utilizatori și a devenit standard în industrie pentru implementarea aplicațiilor de networking.

Acesta are două scopuri extrem de importante. În primul rând, gestionează conexiunile și protocoalele de internet, acestea fiind deja implementate. Astfel, nu este necesară scrierea manuală a codului ce ține de protocoale de comunicare cum ar fi TCP, HTTP sau, în cazul nostru, WebSockets. În al doilea rând, acesta facilitează executarea de operațiuni I/O în mod asincron și non-blocant. Astfel, evităm gestionarea manuală a thread-urilor și ne asigurăm că serverul poate răspunde la cereri de la mai mulți utilizatori în același timp.

Netty reușește acest lucru printr-o implementare a șablonului de proiectare *reactor*. La bază, tot acest sistem este *event-driven*. Netty monitorizează mai multe „canale” (conexiuni), ascultând pentru evenimente. Atunci când are loc un eveniment (de obicei o operațiune de citire/scriere), sunt apelate *handler*-ele corespunzătoare, pentru a procesa evenimentul. Prin această abordare, pot exista mai multe conexiuni, fiecare cu multiple operațiuni ce trebuie efectuate, iar Netty gestionează toate aceste operațiuni într-un mod eficient și integrat.

3.3 Protocol de comunicare

Pentru a facilita comunicarea între clienți și server a trebuit să alegem un protocol de comunicare. Criteriul principal pentru alegerea acestuia a fost dorința de a dezvolta o aplicație de comunicare *în timp real*. Astfel, protocolul de comunicare pe care l-am ales este **WebSockets**. WebSocket permite o comunicare persistentă, full-duplex, între client și server, pentru schimbul de date în timp real. Spre deosebire de alte protocoale, spre exemplu, HTTP, acesta menține o conexiune continuă, fiind mai eficient pentru actualizări frecvente și instantanee. HTTP nu ar fi fost adecvat deoarece acesta nu păstrează o conexiune deschisă. Acesta trimite un request, primește un răspuns, după care conexiunea este întreruptă, trebuind să fie restabilită pentru o cerere nouă. Folosind WebSocket, putem stabili de la început o conexiune între client și server, după care conexiunea va fi folosită pentru a transmite direct doar conținutul efectiv al mesajelor, fără a fi necesară reinițializarea conexiunii de fiecare dată.

3.4 Comunicarea între server și client

Obiecte DTO

Comunicarea între server și client se face folosind obiecte de tip **Data Transfer Object** (DTO). DTO-urile sunt clase care conțin informații cu privire la request-uri și răspunsuri. Pentru request-uri, acestea conțin toate datele de care ar avea nevoie server-ul pentru a îndeplini cererea. Pentru răspunsuri, acestea pot fi un simplu mesaj sau un obiect mai complex. Spre exemplu, pentru o cerere de login, se trimite către server un DTO care conține username-ul și parola introduse de utilizator în UI. Serverul primește acest obiect și execută operațiunile necesare, după care trimite un DTO ce conține un răspuns. DTO-urile sunt folosite și pentru a primi informații de la server. Spre exemplu, atunci când un utilizator cere mesajele dintr-un anumit chat, server-ul execută o cerere către baza de date și apoi trimite către client un DTO ce conține informațiile despre mesajele din conversație.

Serializare/Deserializare

Evident, nu putem trimite direct un obiect Java prin internet. Astfel, avem nevoie de *serializare/deserializare*. Mai concret, avem nevoie de un mod de a codifica informația într-un format mai ușor de trimis, respectiv un mod de a transforma informația aceasta înapoi într-un obiect software. În acest scop s-au folosit fișiere JSON și librăria **Jackson**. Cu ajutorul librăriei Jackson, toate obiectele pe care trebuie să le trimitem sunt transformate în text simplu care urmează formatul unui fișier JSON. Acest lucru este făcut automat de librărie, nefiind necesară o codificare și decodificare manuală a obiectelor.

3.5 Comunicarea între server și baza de date

Comunicarea între server și baza de date este facilitată de obiecte de tip **Data Access Object (DAO)**. Acestea sunt obiectele care comunică efectiv cu baza de date. Ele abstractizează procesul de comunicare cu baza de date și oferă o interfață pentru comunicarea cu aceasta. Astfel, partea de business logic este separată de logica ce ține de conexiunea și comunicarea cu baza de date. De asemenea, acestea centralizează logica pentru baza de date, organizând codul și făcând mentenanța, debugging-ul și scaling-ul semnificativ mai ușor.

3.6 Integrarea Inteligenței Artificiale (AI)

Pentru a extinde capacitățile standard ale unui aplicații de comunicare, am integrat capabilități de procesare a limbajului natural prin utilizarea a două Large ALanguage Models (LLM). Sistemul interoghează un serviciu AI intern complet decuplat, găzduit prin platforma Ollamacu o abordare de tip Multi-Model Routing, pentru că serverul selectează modelul optim în funcție de complexitatea cererii:

- qwen2.5:7b – Utilizat pentru funcția de AI Polish. Fiind un model mai mare (7 miliarde de parametrii), are capacități superioare de generare creativă și respectare a instrucțiunilor complexe .
- gemma3:4b – Utilizat pentru funcția de Recomandare de Grupuri. Fiind un model mai compact și rapid (4 miliarde de parametrii), este ideal pentru sarcini de clasificare, analiză semantică a cuvintelor cheie și procesare rapidă a profilului utilizatorului.

Capabilitățile de inteligență artificială sunt concentrate în două direcții principale de asistență:

Asistentul de Scriere (AI Polish) (qwen2.5:7b)

Această funcție permite utilizatorilor să își corecteze gramatical sau să își rafineze tonul mesajelor înainte de a le expedia. Când utilizatorul apasă butonul specificat în ChatPanel, clasa ChatPresenter generează un obiect AIPolishReq ce conține textul brut și îl trimite prin intermediul clientului. Serverul Netty preia textul, construiește un prompt detaliat și trimite o cerere către instanța Ollama, solicitând explicit utilizarea modelului qwen2.5:7b. După generarea textului corectat, serverul îl împachetează într-un AIPolishRes și îl returnează clientului, care înlocuiește dinamic textul din interfață.

Recomandări Inteligente de Grupuri (gemma3:4b)

Pentru a facilita socializarea și descoperirea de noi comunități, aplicația folosește AI-ul ca pe un motor de recomandare semantică. În panoul de interese (InterestsPanel), utilizatorul își poate defini preferințele. InterestsPresenter trimite un AIRecommendGroupsReq prin intermediul clientului. Backend-ul extrage din baza de date lista grupurilor publice disponibile. Apoi, direcționează o cerere către modelul **gemma3:4b**, cerându-i să evalueze rapid compatibilitatea dintre interesele utilizatorului și descrierile grupurilor. Modelul returnează cele mai relevante sugestii, care sunt ulterior afișate în interfața grafică.

3.7 Interfața cu utilizatorul și arhitectura MVP (Model-View-Presenter)

Arhitectura MVP (Model-View-Presenter)

Frontend-ul aplicației client a fost dezvoltat sub forma unei aplicații desktop native, utilizând biblioteca Java Swing. Pentru a menține o ordine generală în cod și pentru a permite o dezvoltare ulterioară mai facilă a aplicației am utilizat șablonul de proiectare arhitectural MVP (Model-View-Presenter). Acesta asigură o decuplare strictă a responsabilităților și permite testarea automată, împărțind interfața utilizatorului în trei straturi distincte, fiecare cu un rol clar definit:

- **View (Interfața Grafică):** Reprezintă exclusiv elementele vizuale ale aplicației, în cazul proiectului nostru componentele View fiind clasele de tip panou (Panel). Acesta are caracteristica principală de a fi pasiv (Passive View), deoarece nu conține niciun fel de business logic și nu comunică direct cu Model, ci doar cu Presenter. Astfel, singurul său rol este să afișeze date pe ecran, exact cum îi sunt date de Presenter, și să capteze interacțiunile cu utilizatorul, pe care le pasează mai departe către Presenter.
- **Presenter (Prezentatorul):** Este intermediarul și orchestratorul acestei arhitecturi, clasele de tip Presenter conținând logica aplicației client. Acesta are responsabilitatea să preia input-ul de la View, să îl analizeze, să interogheze sau să actualizeze Model-ul, să proceseze răspunsul primit și să dicteze cum să se actualizeze View-ul.
- **Model (Modelul de Date și Rețea):** Reprezintă starea aplicației, datele și mecanismele de comunicare cu serverul, fiind reprezentat de clasele WebSocketClient, WebSocketClientHandler și de Obiectele de Transfer de Date (DTO). Acesta gestionează conexiunea TCP Websocket, decodarea JSON și menținerea listelor interne de date.

Structura Interfeței Grafice

Interfața aplicației Client este contruită respectând un design minimalist și funcțional cu o navigare de tip Single Page Application (SPA), panourile schimbându-se dinamic în cadrul unei singure ferestre principale.

Pagina de Autentificare (LoginPanel)

- Formular centrat cu câmpuri pentru Username și Parolă.
- Design curat, orientat pe funcționalitate, cu suport pentru acțiuni la apăsarea tastei „Enter”.
- Zonă dedicată pentru afișarea mesajelor de feedback asincrone.
- Buton pentru navigarea rapidă către ecranul de înregistrare a utilizatorilor noi.

Pagina de Înregistrare (RegisterPanel)

- Formular cu validare a câmpurilor (username, parolă).
- Feedback vizual imediat în cazul câmpurile sunt incomplete.
- Design vizual consistent cu pagina de autentificare, facilitând o tranziție fluidă între cele două ecrane de acces.

Fereastra Principală de Comunicare (ChatPanel)

- **Bara laterală stânga (Sidebar):** Afișează lista dinamică a chat-urilor active (mesaje directe și grupuri).
- **Zona centrală de chat:** Prezintă istoricul cronologic al conversației selectate. Include un design distinct pentru a diferenția mesajele expediate de cele primite.
- **Zona de introducere:** Câmp text (TextArea) situat în partea inferioară, însoțit de butonul de expediere ("Send").
- **Acțiuni suplimentare:** Acces către corectarea gramaticală a textului folosind Inteligența Artificială (*AI Polish*).

Tab-ul de Prieteni (FriendsPanel)

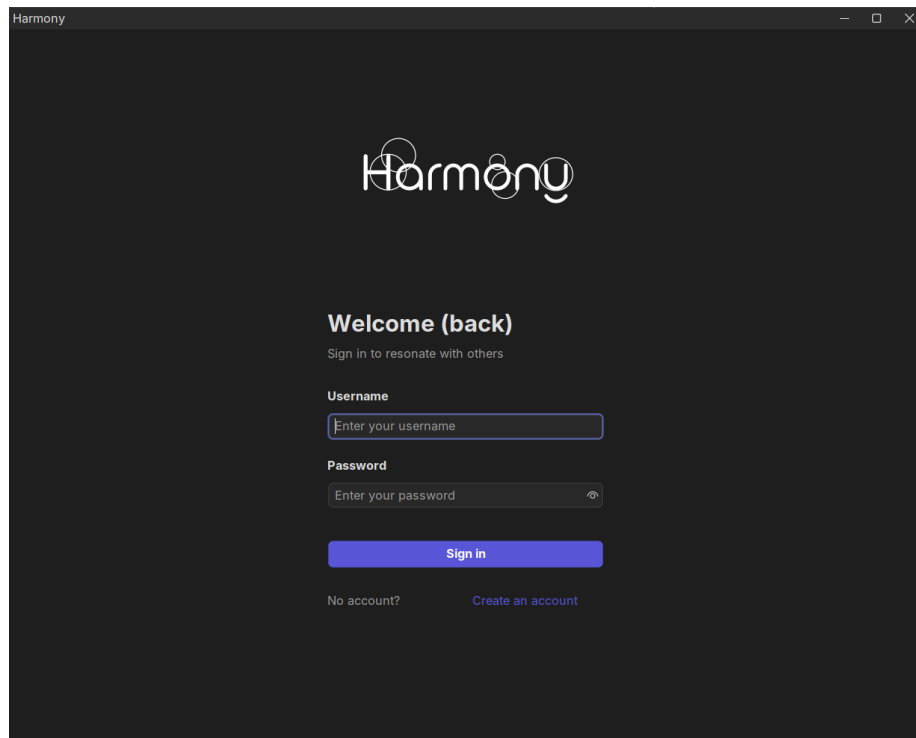
- Interfață cu liste separate pentru gestionarea relațiilor: *Prieteni*, *Cereri de Prietenie Primite*.
- Câmp de căutare pentru identificarea și adăugarea rapidă a noilor utilizatori prin *Username*.
- Butoane de acțiune contextuală în funcție de lista selectată.

Tab-ul de Interese (InterestsPanel)

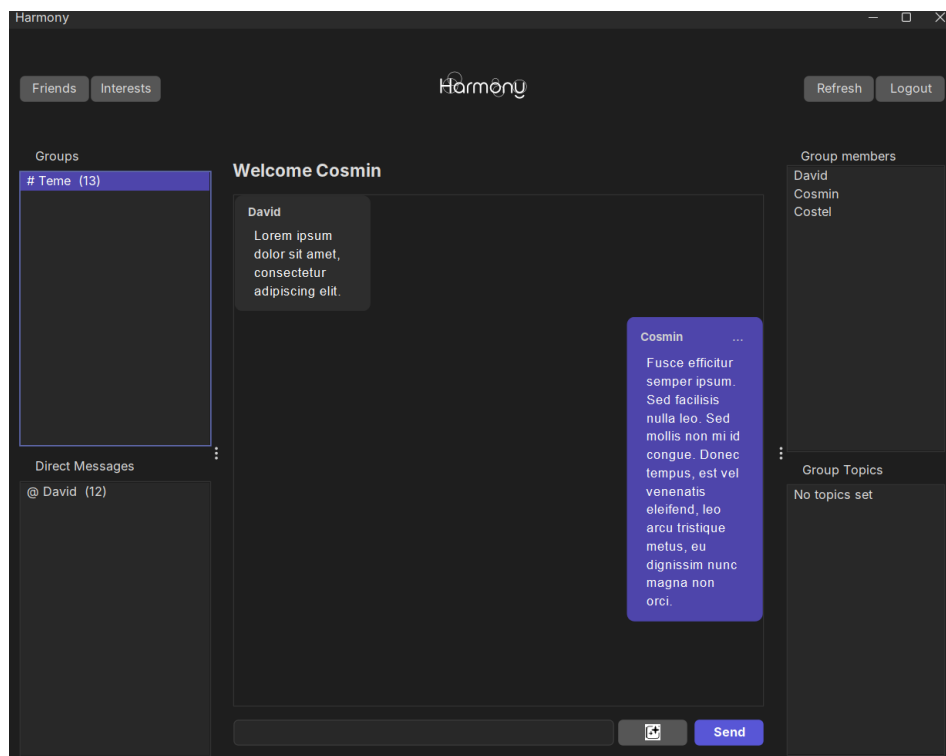
- Interfață centralizată pentru personalizarea profilului utilizatorului.
- Afișarea dinamică a "Top Interese" la nivel global.
- Zonă dedicată interogării serviciului AI pentru a genera și a recomanda automat grupuri de discuții relevante pe baza subiectelor selectate de utilizator.

3.8 Prezentarea interfeței grafice

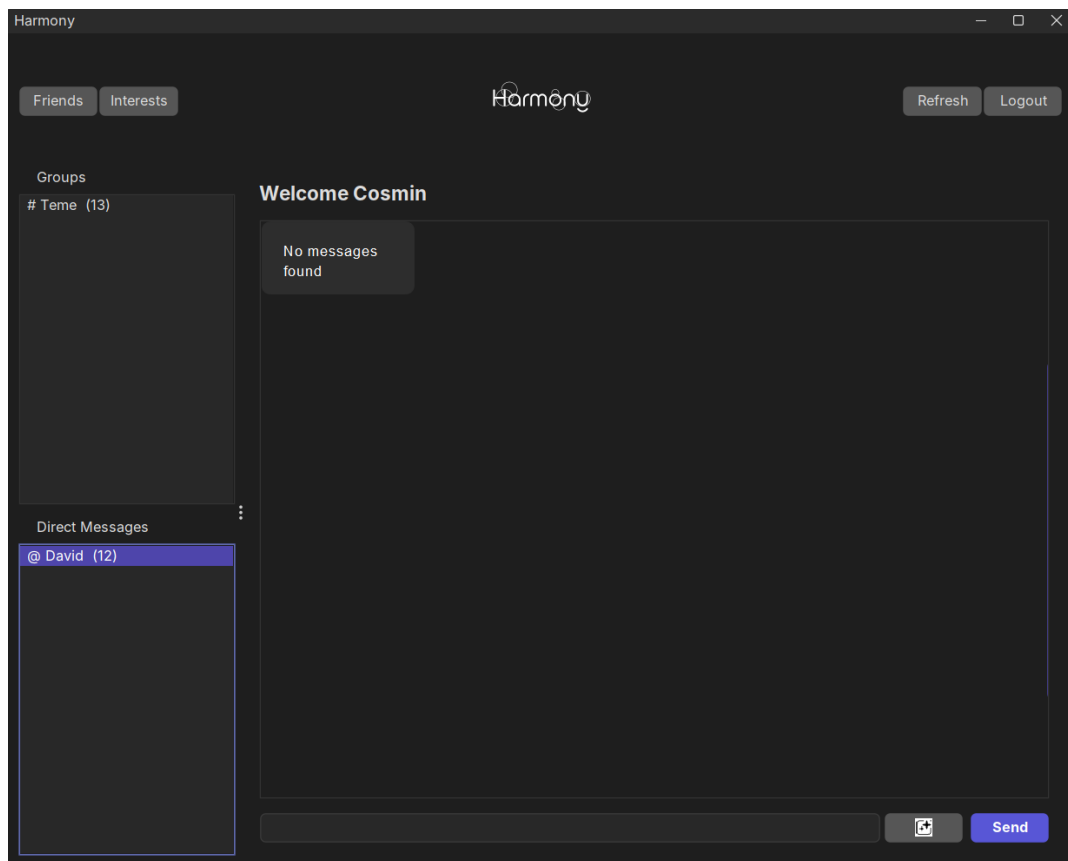
Pagina de login/sign-in



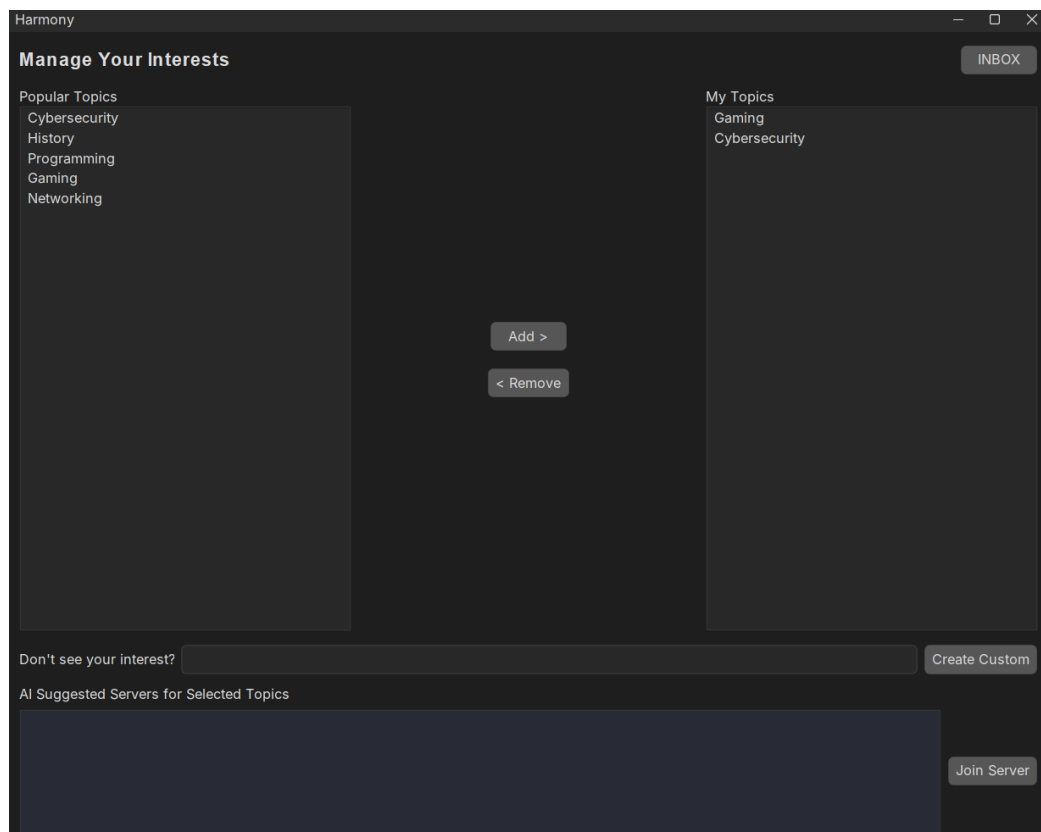
Group Chat



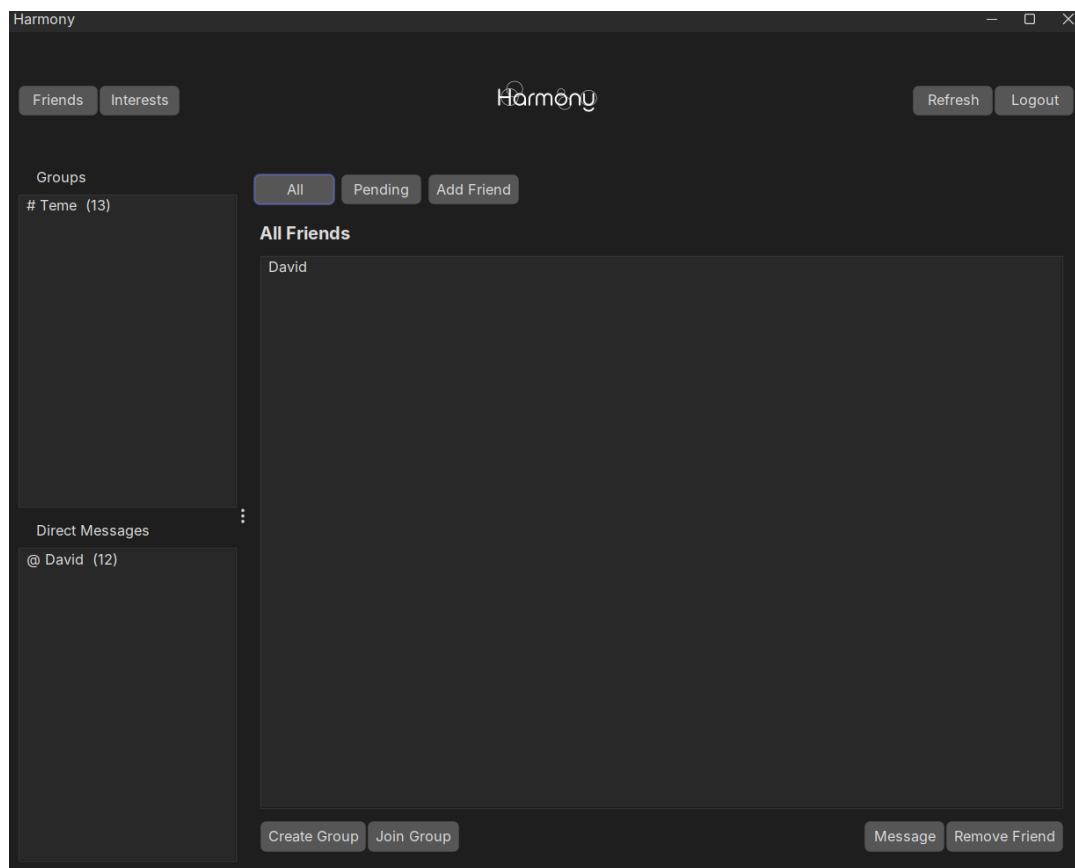
DM gol



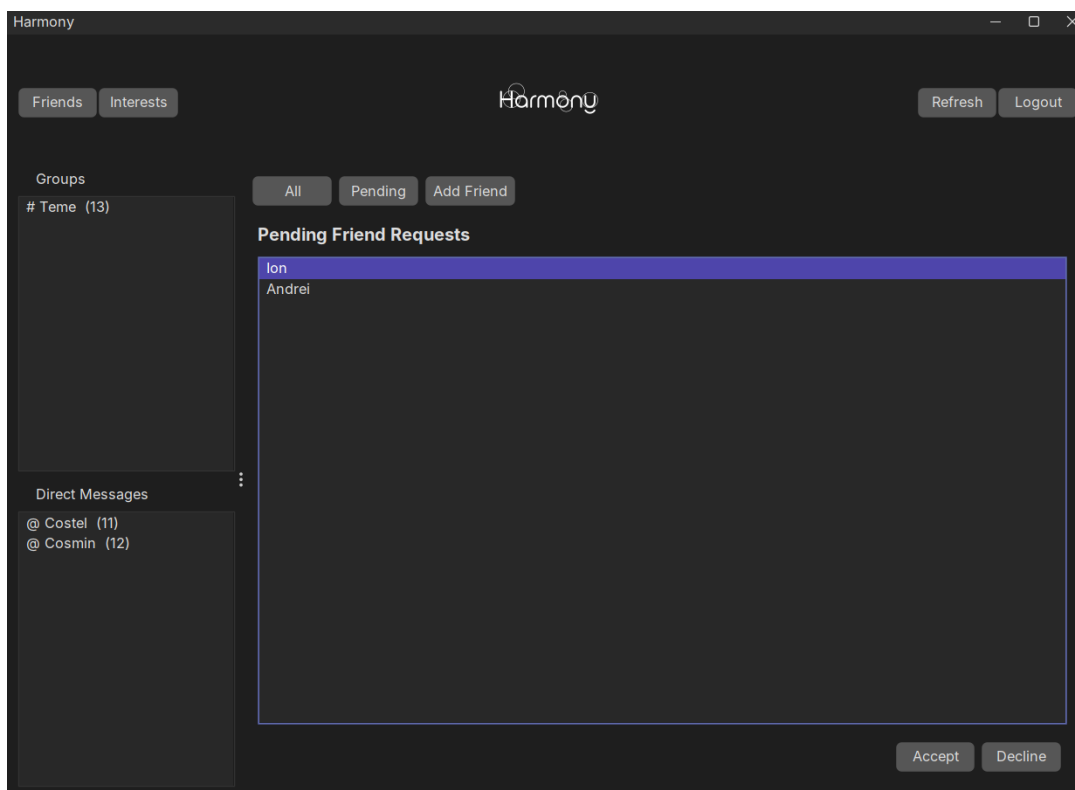
Interfața pentru interese



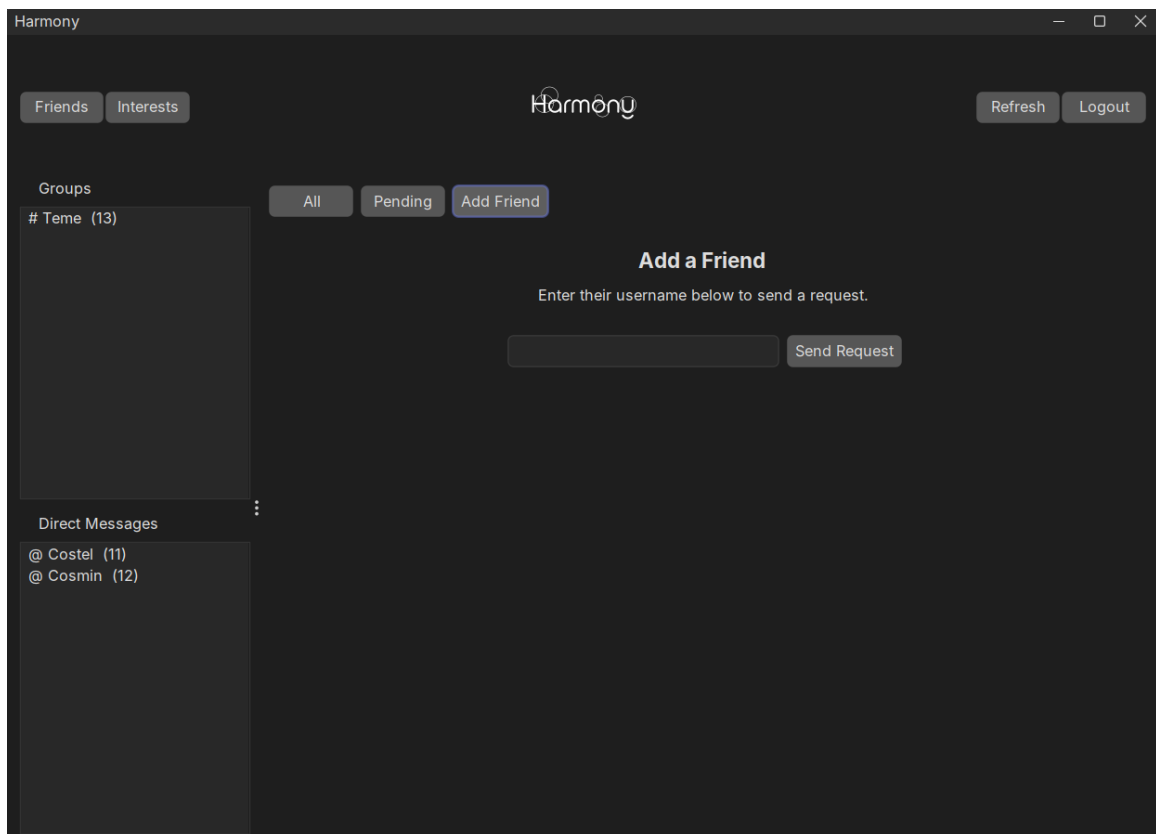
Lista de prieteni



Lista de cereri de prietenie



Adăugarea de prieteni



3.9 Diagrame UML

Diagramă de secvență

Această diagramă de secvență vizualizează procesul de trimitere a unui mesaj. Acesta ilustrează interacțiunea utilizatorului cu *clientul*, comunicarea dintre acesta și *server*, precum și primirea mesajului de către alt utilizator, în timp real.

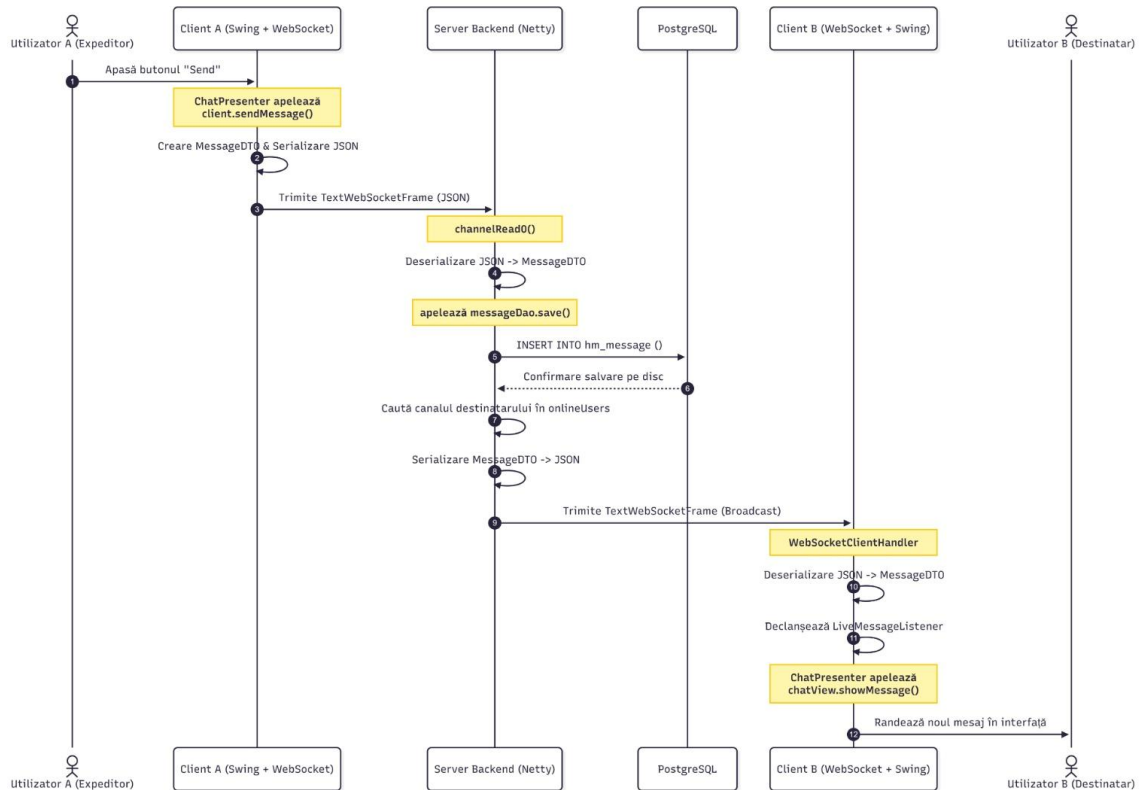
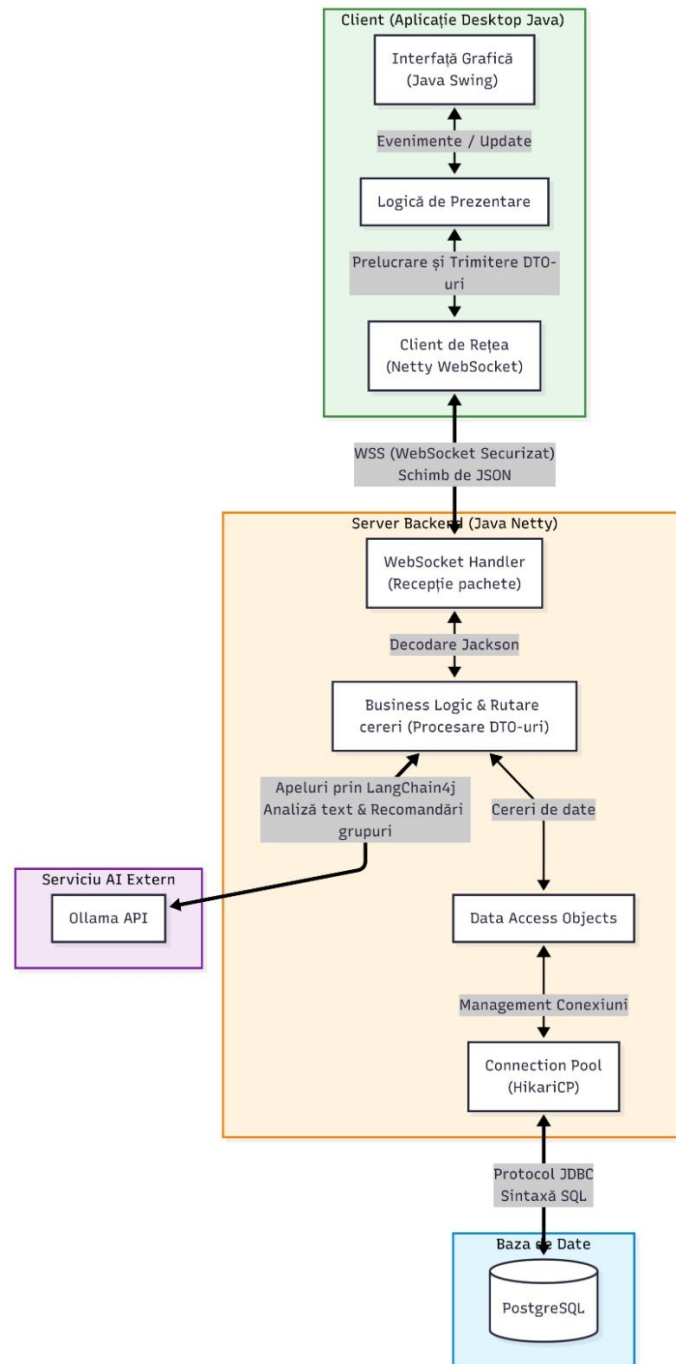


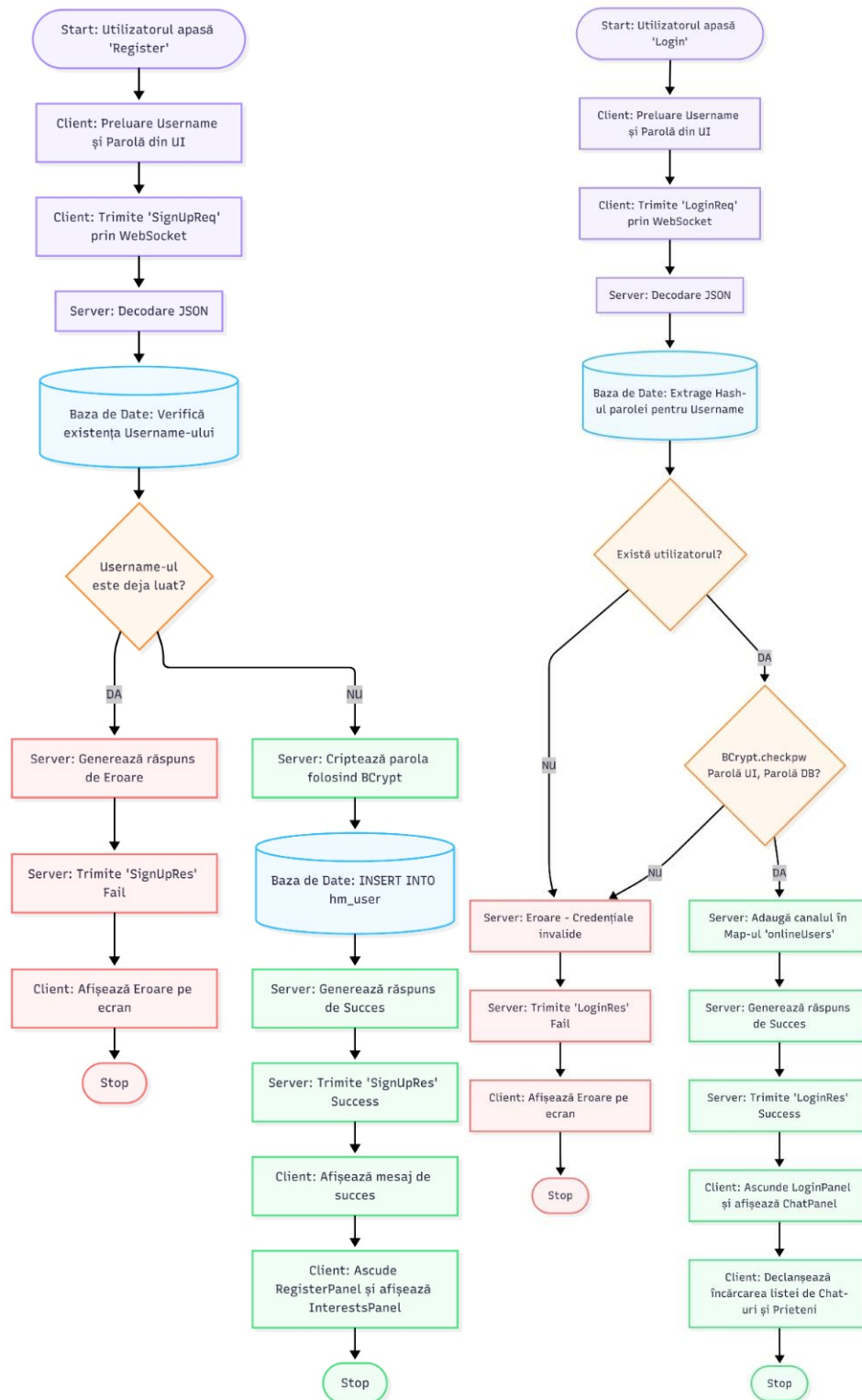
Diagrama de componente

Diagrama de componente ilustrează toate componentele software ale aplicației și modurile principale prin care acestea interacționează între ele. Componentele sunt clientul, serverul, serviciul AI extern și baza de date.



Diagrame use-case

Diagramele use-case vizualizează lanțul de întâmplări ce are loc în urma unei anumite acțiuni ale utilizatorului. Astfel, prezentăm o diagramă pentru procesul de Sign-Up și o diagramă pentru procesul de Sign-In.

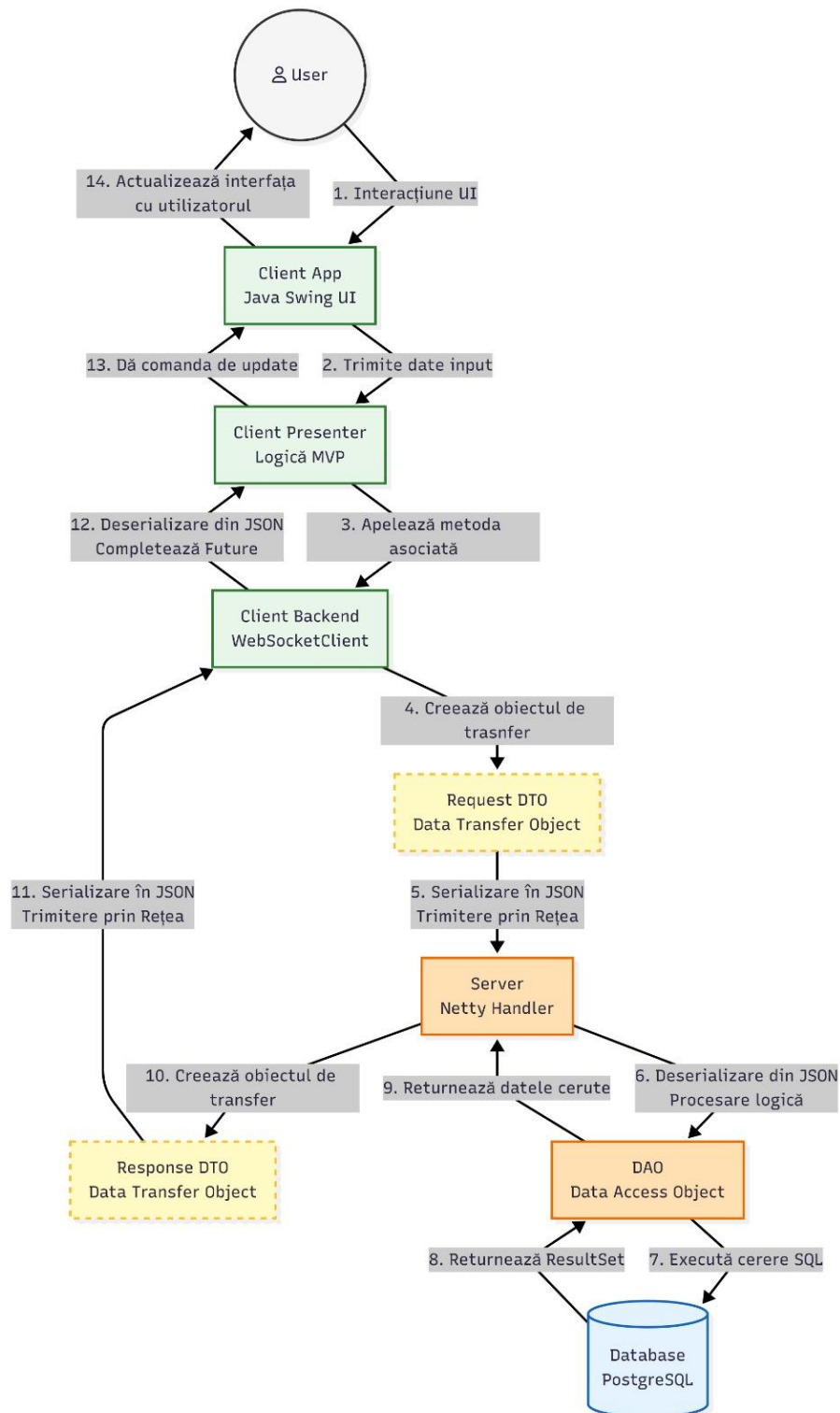


Sign-Up

Sign-In

Diagramă de use-case general

Această diagramă descrie modul general de funcționare al aplicației. Modul în care sunt generate evenimente, modul în care clientul comunică cu serverul și serverul cu baza de date, precum și modul în care informațiile ajung înapoi la client și sunt afișate utilizatorului.



4. Testare

Aplicația dispune de o suită de testare robustă, împărțită în **Teste Unitare** (*Unit Tests*) și **Teste de Integritate** (*Integration Tests*). Scopul principal este garantarea stabilității arhitecturii de tip Client-Server, a decodării pachetelor WebSocket și a logicii interfeței grafice (MVP).

4.1 Tehnologii și Biblioteci

- **JUNIT 5** (Jupiter): Motorul principal de testare.
- **Mockito**: Mimează (*Mock*) interacțiunile cu baza de date (ResultSet, PreparedStatement), interfața grafică (Swing Panels) și traficul de rețea în testele unitare.
- **Testcontainers**: Ridică containere *Docker* reale de *PostgreSQL* pentru testarea pe o bază de date reală fără a afecta mediul de producție în testele de integrare.

4.2 Teste Unitare (Unit Tests)

Aceste teste validează componente individuale ale aplicației în izolare completă, folosind Mockito pentru a simula restul sistemului.

Testele pentru Obiectele de Acces ale Bazei de Date (DAO - Data Access Objects)

Verifică logica de mapare a obiectelor Java către și dinspre baza de date, simulând un server SQL.

- **UserDaoTest** (5 teste): Validează verificarea existenței utilizatorilor, logarea, crearea conturilor, adăugarea de discuții cu mesaje directe între utilizatori și preluarea prietenilor pentru un utilizator. Mimează sistemul de hashing BCrypt și rezultatele SQL.
- **ChatDaoTest** (4 teste): Testează recuperarea listei de chat-uri, adăugarea grupurilor, verificarea existenței unui utilizator și găsirea subiectelor (topicurilor) de discuție pentru un chat.
- **MessageDaoTest** (4 teste): Verifică recuperarea istoricului, inserarea, editarea și ștergerea mesajelor.
- **InterestsDaoTest** (3 teste): Validează adăugarea, ștergerea și extragerea de interese.

Testele pentru Obiectele de Transfer de Date (DTO – Data Transfer Objects)

RequestDTOTest (14 teste), **ResponseDTOTest** (12 teste) și **CoreDTOTest** (3 teste) asigură că toate obiectele de transfer a datelor sunt serializate și deserializate corect (din obiecte Java în string-uri JSON și invers), folosind ObjectMapper-ul din biblioteca Jackson.

Testele de Backend (Netty Server și Client)

Evaluează robustețea sistemului prin testarea părții de Networking (*Netty*) care asigură comunicarea Client-Server.

- *WebSocketServerHandlerTest* (5 teste): validează funcția *channelRead0* prin interceptarea frame-urilor text (JSON) și verificarea corectitudinii rutării pachetelor către metodele corespunzătoare și a blocării utilizatorilor neautentificați. Canalul de rețea (*ChannelHandlerContext*) este mimat pentru a evita pornirea unui server real în timpul testului unitar.
- *WebSocketClientHandlerTest* (6 teste): testează motorul care procesează frame-urile venite de la server. Astfel, pentru cicluri de tip Cerere-Răspuns (*Request-Response*) se verifică blocarea corectă a firului de execuție și deblocarea acestuia la primirea JSON-ului corespunzător, iar pentru evenimente Live (notificare asincronă), testul trebuie să demonstreze că interfața de tip *Live Listener* este notificată instantaneu.
- *WebSocketClientTest* (5 teste): verifică dacă obiectele Java din aplicație sunt transformate corect în JSON și trimse pe canalul *Netty*.

Testele de Frontend (Arhitectura MVP – Presenters)

Verifică logica aplicației Swing (găzduită în clasele de tip Presenter) fără a rula (randa) efectiv ferestre pe ecran. Ele mimează elementele de UI și interceptează firele de execuție în fundal (*SwingWorker*).

- *LoginPresenterTest* (4 teste) și *RegisterPresenterTest* (3 teste): testează dacă datele preluate din câmpurile de text generează corect un apel de rețea și dacă interfața grafică reacționează afișând erori sau schimbând fereastra în funcție de succesul sau eșecul operațiunii.
- *ChatPresenterTest* (3 teste): validează funcționalitatea de trimitere a mesajelor prin injectarea unor date de test pentru a simula selectarea unei conversații. Testul monitorizează firul de execuție și comportamentul aplicației în diferite scenarii, precum când mesajul este gol, când conține text valid sau când se utilizează funcția *AI Polish*.
- *FriendsPresenterTest* (2 teste), *InterestsPresenterTest* (5 teste) și *InboxPresenterTest* (2 teste): verifică încărcarea asincronă a datelor de pe server și actualizarea dinamică a listelor din interfața grafică. În Swing, actualizarea interfeței trebuie să se facă pe Event Dispatch Thread (EDT), în timp ce preluarea datelor se face pe un fir de execuție separat, astfel testele validează că Presenter-ul orchestrează corect acești pași:
 - Solicită View-ului să afișeze un indicator de încărcare (loading).
 - Apelează clientul WebSocket pentru a prelua lista de date de pe rețea.

- Solicită View-ului să șteargă datele vechi și să populeze listele cu noile date primite.
- `ClientPresenterTest` (2 teste): asigură funcționarea corectă a navigării între ecranele aplicației.

4.3 Teste de Integrare (Integration Tests)

Aceste teste verifică sistemul exact cum va rula în producție: prin rețea TCP reală și cu baze de date reale, folosind module Docker temporare.

`DaoIntegrationTest`

Validează sintaxa SQL și funcționarea driverului JDBC.

Mod de funcționare: Ridică un container *Docker* cu *PostgreSQL 15*, creează dinamic tabelele necesare și apelează metodele reale din *UserDao* pentru a insera un utilizator și a verifica hash-ul parolei direct din baza de date. La finalul testului, containerul este distrus.

`ServerIntegrationTest`

Validează Pipeline-ul *Netty* din *WebSocketServerInitializer* și testează codificarea și decodificarea la nivel de rețea.

Mod de funcționare: Pornește serverul *Netty* pe un port de testare, folosește un client *HTTP Java* nativ pentru a negocia un *handshake* de *WebSocket*, apoi trimite un *JSON* invalid sau pentru un utilizator inexistent și se așteaptă la un răspuns *LoginRes* negativ valid.

`ComplexFlowIntegrationTest`

Simulează un scenariu complet și complex de utilizare a aplicației de la un capăt la celălalt (End-to-End).

Scenariul:

- `TestContainers` ridică o bază de date *PostgreSQL* și o populează cu 2 utilizatori și un grup.
- Serverul *Netty* este pornit pe portul de test și este injectat cu conexiunea la baza de date *Docker* temporară.
- Se creează instanțe reale de *WebSocketClient* pentru fiecare utilizator, iar acestea trec prin procesul complet de *handshake* *WebSocket*.
- Utilizatorii trimit pachete *LoginReq* prin rețea și devin online în server.
- Un utilizator trimite un mesaj text pe grup.

- Serverul primește mesajul, îl salvează în baza de date *Docker* și determină că celălalt utilizator face parte din acel chat, astfel va retrimite mesajul (broadcast) către sesiunea acestuia.
- Firul de execuție de test așteaptă ca listenerul *LiveMessageListener* al clientului destinatar să detecteze primirea *JSON*-ului.
- Se execută o cerere de tip *SELECT* direct pe baza de date *Docker* pentru a verifica dacă mesajul trimis a fost salvat cu succes pe disc

4.4 Integrare continuă (GitHub Actions)

Proiectul este compatibil cu *GitHub Actions*, permițând rularea comenzii *mvn test* automat la fiecare *Commit* sau *Pull Request*. Prin ridicarea automată a containerelor *Docker* în cloud-ul *GitHub* pe o mașină virtuală care rulează ultima versiune de Ubuntu, orice cod care nu are comportamentul așteptat va fi interceptat de suita de teste.

5. Raport privind folosirea instrumentelor AI în timpul dezvoltării

În cadrul ciclului de dezvoltare al acestui proiect, instrumentele bazate pe Inteligența Artificială (AI) au fost integrate pentru a optimiza fluxul de lucru și a reduce timpul alocat sarcinilor de rutină.

În fazele incipiente ale proiectului, mai precis în etapa de analiză și design, a fost utilizat instrumentul Perplexity AI cu scopul principal de eficientizare a procesului de documentare pentru:

- **Arhitectura Generală:** AI-ul a fost folosit pentru a identifica și sintetiza resurse generale, articole tehnice și bune practici referitoare la proiectarea arhitecturii Client-Server a aplicației.
- **Abordarea Tehnică:** Perplexity AI a ajutat la evaluarea diverselor soluții tehnologice, precum utilizarea Netty pentru rețea și PostgreSQL pentru baza de date, contribuind la conturarea strategiei optime de abordare a sistemului înainte de partea efectivă de implementare.

Pe parcursul etapei de programare, Google Gemini a fost adoptat pentru asistarea scrierii de cod în următoarele situații:

- **Sarcini Repetitive:** În dezvoltarea aplicației Java am întâlnit situații în care trebuia să scriem un volum mare de cod predictibil și repetitiv (cunoscut sub numele de boilerplate code), întrucât Java este un limbaj cu sintaxă verbosă. Gemini a fost folosit pentru generarea rapidă a unora din clasele de tip DTO (Data Transfer Objects), care presupune maparea câmpurilor, generarea de constructori și a metodelor de tip getter/setter, permițând concentrarea pe logica de business.
- **Generarea de Teste:** Datorită structurii repetitive a anumitor teste, precum testele unitare pentru validarea rutelor DTO sau verificările standard din arhitectura MVP (Model-View-Presenter), Gemini a fost utilizat pentru generarea de mock-uri și schelete de cod pentru testele automate. Acest lucru a accelerat semnificativ procesul de acoperire a codului în procesul de testare.

În timpul dezvoltării, unele provocări tehnice specifice au fost soluționate prin folosirea instrumentelor AI. În anumite instanțe în care erorile (excepțiile de rețea, blocajele de firele de execuție sau erorile SQL) au fost greu de izolat, Gemini a fost utilizat pentru a analiza log-urile aplicației (stack traces). Această abordare a oferit adesea o altă perspectivă și a condus la identificarea blocajelor. În plus, pentru exemple specifice privind sintaxa limbajului Java și modul de corect de apelare a funcțiilor din librării externe (Netty, Jackson, HikariCP, JUnit), AI-ul a furnizat fragmente de cod

precise, care au facilitat înțelegerea modului de configurare și instanțiere a anumitor obiecte.